10. 재귀 증명 (Recursive Proofs)

목차

- 10.1 재귀 증명 개념
- 10.2 VerifierCircuitTarget 구조
- 10.3 재귀 검증 회로 구성
- 10.4 verify_proof 함수 분석
- 10.5 재귀 증명 예제
- 10.6 조건부 재귀 검증

10.1 재귀 증명 개념

재귀 증명이란?

재귀 증명(Recursive Proof)은 **증명의 증명**을 생성하는 기법입니다. 즉, 어떤 증명이 유효한지를 검증하는 과정 자체를 회로로 구성하고, 그 검증 과정에 대한 증명을 생성합니다.

재귀 증명의 필요성

1. 증명 크기 압축

여러 개의 증명을 하나의 증명으로 압축할 수 있습니다. 예를 들어:

- 100개의 트랜잭션 증명 → 1개의 배치 증명
- 여러 단계의 계산 증명 → 1개의 최종 증명

2. 검증 시간 단축

재귀적으로 증명을 압축하면 최종 검증자는 하나의 증명만 검증하면 됩니다:

```
증명1, 증명2, ..., 증명N

↓ (재귀 증명)

단일 증명

↓ (빠른 검증)

검증 완료
```

3. 증명 체인 구성

블록체인처럼 증명을 체인으로 연결할 수 있습니다:

```
블록1 증명 → 블록2 증명 → 블록3 증명 → ... → 최종 증명
```

재귀 증명의 동작 원리

plonky2에서 재귀 증명은 다음과 같이 동작합니다:

1. **내부 증명(Inner Proof)**: 검증하고자 하는 원본 증명
2. **검증 회로**: 내부 증명을 검증하는 로직을 회로로 구현
3. **외부 증명(Outer Proof)**: 검증 회로에 대한 증명 생성

```
// 내부 증명 생성
let  inner_proof = inner_circuit.prove(inner_witness)?;


// 재귀 검증 회로 구성
```

```rust
let mut builder = CircuitBuilder::new(config);
let proof_target = builder.add_virtual_proof_with_pis(&inner_common_data);
let verifier_data_target = builder.add_virtual_verifier_data(cap_height);


// 검증 로직을 회로에 추가
builder.verify_proof::<InnerConfig>(&proof_target, &verifier_data_target, &inner


// 외부 증명 생성
let outer_circuit = builder.build::<OuterConfig>();
let outer_proof = outer_circuit.prove(outer_witness)?;
```

10.2 VerifierCircuitTarget 구조

정의

`VerifierCircuitTarget` 은 재귀 회로 내에서 검증자 데이터를 표현하는 구조체입니다:

```rust
/// The `Target` version of `VerifierCircuitData`, for use inside recursive circ
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct VerifierCircuitTarget {
/// A commitment to each constant polynomial and each permutation polynomial.
pub constants_sigmas_cap: MerkleCapTarget,
/// A digest of the "circuit" (i.e. the instance, minus public inputs),
```

```
/// which can be used to seed Fiat-Shamir.
pub  circuit_digest: HashOutTarget,
}
```

필드 설명

1. constants_sigmas_cap

- **타입**: `MerkleCapTarget`
- **역할**: 상수 다항식과 순열 다항식(sigma)에 대한 Merkle Tree의 Cap
- **용도**: 내부 회로의 전처리된 데이터를 검증

```
// Merkle Cap은 Merkle Tree의 상위 레벨 해시들의 집합
pub  struct MerkleCapTarget(pub Vec<HashOutTarget>);
```

2. circuit_digest

- **타입**: `HashOutTarget`
- **역할**: 회로 구조의 해시 다이제스트
- **용도**: Fiat-Shamir 변환에서 회로 식별자로 사용

```
// HashOutTarget은 해시 출력을 나타내는 Target들의 배열
pub  struct HashOutTarget {
pub  elements: [Target; 4], // Goldilocks Field에서는 4개 원소
}
```

VerifierCircuitTarget 생성

재귀 회로에서 `VerifierCircuitTarget` 을 생성하는 방법:

```
impl<F: RichField + Extendable<D>, const D: usize> CircuitBuilder<F, D> {
pub fn add_virtual_verifier_data(&mut self, cap_height: usize) -> VerifierCir
VerifierCircuitTarget {
constants_sigmas_cap: self.add_virtual_cap(cap_height),
circuit_digest: self.add_virtual_hash(),
}
}
}
```

Witness 설정

생성된 `VerifierCircuitTarget` 에 실제 검증자 데이터를 설정:

```
// VerifierCircuitTarget 생성
let verifier_data_target = builder.add_virtual_verifier_data(cap_height);


// Witness 설정
let mut pw = PartialWitness::new();
pw.set_cap_target(
&verifier_data_target.constants_sigmas_cap,
&inner_verifier_data.constants_sigmas_cap,
```

```
)?;
pw.set_hash_target(
verifier_data_target.circuit_digest,
inner_verifier_data.circuit_digest,
)?;
```

10.3 재귀 검증 회로 구성

전체 흐름

재귀 검증 회로를 구성하는 단계:

1. **증명 Target 추가**: 검증할 증명을 회로의 입력으로 추가
2. **검증자 데이터 Target 추가**: 검증에 필요한 데이터 추가
3. **검증 로직 추가**: `verify_proof` 함수로 검증 회로 구성
4. **회로 빌드**: 최종 회로 생성

1. 증명 Target 추가

```
pub fn add_virtual_proof_with_pis(
&mut self,
common_data: &CommonCircuitData<F, D>,
) -> ProofWithPublicInputsTarget<D> {
let proof = self.add_virtual_proof(common_data);
```

```
let public_inputs = self.add_virtual_targets(common_data.num_public_inputs);
ProofWithPublicInputsTarget {
proof,
public_inputs,
}
}
```

ProofWithPublicInputsTarget 구조:

```
pub struct ProofWithPublicInputsTarget<const D: usize> {
pub proof: ProofTarget<D>,
pub public_inputs: Vec<Target>,
}
```

ProofTarget 구조:

```
pub struct ProofTarget<const D: usize> {
pub wires_cap: MerkleCapTarget,
pub plonk_zs_partial_products_cap: MerkleCapTarget,
pub quotient_polys_cap: MerkleCapTarget,
pub openings: OpeningSetTarget<D>,
pub opening_proof: FriProofTarget<D>,
}
```

2. 검증자 데이터 추가

```
let  verifier_data_target = builder.add_virtual_verifier_data(
inner_common_data.config.fri_config.cap_height
);
```

3. 검증 로직 추가

```
builder.verify_proof::<InnerConfig>(
&proof_target,
&verifier_data_target,
&inner_common_data,
);
```

완전한 예제

```
use plonky2::plonk::circuit_builder::CircuitBuilder;
use plonky2::plonk::circuit_data::CircuitConfig;
use plonky2::plonk::config::PoseidonGoldilocksConfig;
use plonky2::iop::witness::PartialWitness;

const D: usize = 2;
type C = PoseidonGoldilocksConfig;
type F = <C as GenericConfig<D>>::F;
```

```rust
fn create_recursive_circuit(
inner_proof: ProofWithPublicInputs<F, C, D>,
inner_verifier_data: VerifierOnlyCircuitData<C, D>,
inner_common_data: CommonCircuitData<F, D>,
) -> Result<CircuitData<F, C, D>> {
let config = CircuitConfig::standard_recursion_config();
let mut builder = CircuitBuilder::<F, D>::new(config);
// 1. 증명 Target 추가
let proof_target = builder.add_virtual_proof_with_pis(&inner_common_data);
// 2. 검증자 데이터 Target 추가
let verifier_data_target = builder.add_virtual_verifier_data(
inner_common_data.config.fri_config.cap_height
);
// 3. 검증 로직 추가
builder.verify_proof::<C>(
&proof_target,
&verifier_data_target,
&inner_common_data,
);
// 4. 회로 빌드
let circuit_data = builder.build::<C>();
Ok(circuit_data)
```

```
}
```

10.4 verify_proof 함수 분석

함수 시그니처

```
impl<F: RichField + Extendable<D>, const D: usize> CircuitBuilder<F, D> {
/// Recursively verifies an inner proof.
pub fn verify_proof<C: GenericConfig<D, F = F>>(
&mut self,
proof_with_pis: &ProofWithPublicInputsTarget<D>,
inner_verifier_data: &VerifierCircuitTarget,
inner_common_data: &CommonCircuitData<F, D>,
) where
C::Hasher: AlgebraicHasher<F>,
{
// 검증 로직
}
}
```

검증 단계

1. 공개 입력 검증

```
assert_eq!(
proof_with_pis.public_inputs.len(),
inner_common_data.num_public_inputs
);
```

공개 입력의 개수가 예상과 일치하는지 확인합니다.

2. 공개 입력 해시 계산

```
let public_inputs_hash = self.hash_n_to_hash_no_pad::<C::InnerHasher>(
proof_with_pis.public_inputs.clone()
);
```

공개 입력들을 해시하여 단일 해시값으로 압축합니다.

3. 챌린지 생성

```
let challenges = proof_with_pis.get_challenges::<F, C>(
self,
public_inputs_hash,
inner_verifier_data.circuit_digest,
inner_common_data,
);
```

Fiat-Shamir 변환을 통해 검증에 필요한 챌린지들을 생성합니다.

4. 상세 검증 수행

```
self.verify_proof_with_challenges::<C>(
&proof_with_pis.proof,
public_inputs_hash,
challenges,
inner_verifier_data,
inner_common_data,
);
```

verify_proof_with_challenges 내부 동작

1. Vanishing Polynomial 평가

```
let  vars = EvaluationTargets {
local_constants,
local_wires,
public_inputs_hash: &public_inputs_hash,
};
```

```
let  vanishing_polys_zeta = eval_vanishing_poly_circuit::<F, D>(
self,
inner_common_data,
challenges.plonk_zeta,
zeta_pow_deg,
vars,
```

```
local_zs,
next_zs,
local_lookup_zs,
next_lookup_zs,
partial_products,
s_sigmas,
&challenges.plonk_betas,
&challenges.plonk_gammas,
&challenges.plonk_alphas,
&challenges.plonk_deltas,
);
```

모든 제약조건을 평가하여 vanishing polynomial 값을 계산합니다.

2. Quotient Polynomial 검증

```
let quotient_polys_zeta = &proof.openings.quotient_polys;
let mut scale = ReducingFactorTarget::new(zeta_pow_deg);
let z_h_zeta = self.sub_extension(zeta_pow_deg, one);

for (i, chunk) in quotient_polys_zeta
.chunks(inner_common_data.quotient_degree_factor)
.enumerate()
{
```

```
let recombined_quotient = scale.reduce(chunk, self);
let computed_vanishing_poly = self.mul_extension(z_h_zeta, recombined_quotient)
self.connect_extension(vanishing_polys_zeta[i], computed_vanishing_poly);
}
```

Quotient polynomial이 올바르게 계산되었는지 검증합니다:

- `vanishing_poly(ζ) = Z_H(ζ) * quotient(ζ)`

3. FRI 검증

```
let merkle_caps = &[
inner_verifier_data.constants_sigmas_cap.clone(),
proof.wires_cap.clone(),
proof.plonk_zs_partial_products_cap.clone(),
proof.quotient_polys_cap.clone(),
];

let fri_instance = inner_common_data.get_fri_instance_target(self, challenges.p

self.verify_fri_proof::<C>(
&fri_instance,
&proof.openings.to_fri_openings(),
&challenges.fri_challenges,
merkle_caps,
&proof.opening_proof,
```

```
&inner_common_data.fri_params,
);
```

FRI 프로토콜을 통해 다항식 opening이 올바른지 검증합니다.

10.5 재귀 증명 예제

기본 재귀 증명 예제

```
use anyhow::Result;
use plonky2::plonk::circuit_builder::CircuitBuilder;
use plonky2::plonk::circuit_data::CircuitConfig;
use plonky2::plonk::config::PoseidonGoldilocksConfig;
use plonky2::iop::witness::{PartialWitness, WitnessWrite};
use plonky2::field::types::Field;

const D: usize = 2;
type C = PoseidonGoldilocksConfig;
type F = <C as GenericConfig<D>>::F;

fn main() -> Result<()> {
// ===== 1단계: 내부 증명 생성 =====
let config = CircuitConfig::standard_recursion_config();
```

```rust
let mut inner_builder = CircuitBuilder::<F, D>::new(config.clone());
// 간단한 회로: a + b = c
let a = inner_builder.add_virtual_target();
let b = inner_builder.add_virtual_target();
let c = inner_builder.add(a, b);
inner_builder.register_public_input(a);
inner_builder.register_public_input(b);
inner_builder.register_public_input(c);
let inner_data = inner_builder.build::<C>();
// Witness 설정 및 증명 생성
let mut inner_pw = PartialWitness::new();
inner_pw.set_target(a, F::from_canonical_u64(3))?;
inner_pw.set_target(b, F::from_canonical_u64(5))?;
let inner_proof = inner_data.prove(inner_pw)?;
println!("내부 증명 생성 완료");
println!("공개 입력: {:?}", inner_proof.public_inputs);
// 내부 증명 검증
inner_data.verify(inner_proof.clone())?;
println!("내부 증명 검증 성공");
// ===== 2단계: 재귀 증명 생성 =====
let mut outer_builder = CircuitBuilder::<F, D>::new(config);
let mut outer_pw = PartialWitness::new();
// 내부 증명을 Target으로 추가
```

```
let proof_target = outer_builder.add_virtual_proof_with_pis(&inner_data.common)
outer_pw.set_proof_with_pis_target(&proof_target, &inner_proof)?;
// 검증자 데이터 추가
let verifier_data_target = outer_builder.add_virtual_verifier_data(
inner_data.common.config.fri_config.cap_height
);
outer_pw.set_cap_target(
&verifier_data_target.constants_sigmas_cap,
&inner_data.verifier_only.constants_sigmas_cap,
)?;
outer_pw.set_hash_target(
verifier_data_target.circuit_digest,
inner_data.verifier_only.circuit_digest,
)?;
// 검증 로직 추가
outer_builder.verify_proof::<C>(
&proof_target,
&verifier_data_target,
&inner_data.common,
);
// 외부 회로 빌드 및 증명 생성
let outer_data = outer_builder.build::<C>();
let outer_proof = outer_data.prove(outer_pw)?;
```

```
println!("재귀 증명 생성 완료");
// 재귀 증명 검증
outer_data.verify(outer_proof)?;
println!("재귀 증명 검증 성공");
Ok(())
}
```

다중 재귀 증명 (Proof Aggregation)

여러 증명을 하나로 집계하는 예제:

```
fn aggregate_proofs(
proofs: Vec<ProofWithPublicInputs<F, C, D>>,
verifier_data: VerifierOnlyCircuitData<C, D>,
common_data: CommonCircuitData<F, D>,
) -> Result<ProofWithPublicInputs<F, C, D>> {
let config = CircuitConfig::standard_recursion_config();
let mut builder = CircuitBuilder::<F, D>::new(config);
let mut pw = PartialWitness::new();
// 각 증명을 검증하는 회로 구성
for proof in proofs {
let proof_target = builder.add_virtual_proof_with_pis(&common_data);
pw.set_proof_with_pis_target(&proof_target, &proof)?;
let verifier_data_target = builder.add_virtual_verifier_data(
```

```
common_data.config.fri_config.cap_height
);
pw.set_cap_target(
&verifier_data_target.constants_sigmas_cap,
&verifier_data.constants_sigmas_cap,
)?;
pw.set_hash_target(
verifier_data_target.circuit_digest,
verifier_data.circuit_digest,
)?;
builder.verify_proof::<C>(&proof_target, &verifier_data_target, &common_data);
}
// 집계 증명 생성
let aggregated_data = builder.build::<C>();
let aggregated_proof = aggregated_data.prove(pw)?;
Ok(aggregated_proof)
}
```

재귀 체인 예제

증명을 체인으로 연결하는 예제:

```
fn create_proof_chain(depth: usize) -> Result<ProofWithPublicInputs<F, C, D>> {
let config = CircuitConfig::standard_recursion_config();
```

```
// 초기 증명 생성
let mut current_proof = create_base_proof(&config)?;
let mut current_vd = get_verifier_data(&current_proof);
let mut current_cd = get_common_data(&current_proof);
// 재귀적으로 증명 체인 생성
for i in 0..depth {
println!("재귀 레벨 {}", i + 1);
let mut builder = CircuitBuilder::<F, D>::new(config.clone());
let mut pw = PartialWitness::new();
// 이전 증명 검증
let proof_target = builder.add_virtual_proof_with_pis(&current_cd);
pw.set_proof_with_pis_target(&proof_target, &current_proof)?;
let verifier_data_target = builder.add_virtual_verifier_data(
current_cd.config.fri_config.cap_height
);
pw.set_cap_target(
&verifier_data_target.constants_sigmas_cap,
&current_vd.constants_sigmas_cap,
)?;
pw.set_hash_target(
verifier_data_target.circuit_digest,
current_vd.circuit_digest,
)?;
```

```
builder.verify_proof::<C>(&proof_target, &verifier_data_target, &current_cd);

// 새로운 증명 생성

let data = builder.build::<C>();

current_proof = data.prove(pw)?;

current_vd = data.verifier_only;

current_cd = data.common;

}

Ok(current_proof)

}
```

10.6 조건부 재귀 검증

조건부 검증 개념

조건에 따라 서로 다른 증명을 검증하는 기능입니다:

```
pub fn conditionally_verify_proof<C: GenericConfig<D, F = F>>(

&mut self,

condition: BoolTarget,

proof_with_pis0: &ProofWithPublicInputsTarget<D>,

inner_verifier_data0: &VerifierCircuitTarget,

proof_with_pis1: &ProofWithPublicInputsTarget<D>,

inner_verifier_data1: &VerifierCircuitTarget,
```

```
inner_common_data: &CommonCircuitData<F, D>,
) where
C::Hasher: AlgebraicHasher<F>,
```

동작 원리

```
// condition이 true면 proof0 검증, false면 proof1 검증
let selected_proof = self.select_proof_with_pis(
condition,
proof_with_pis0,
proof_with_pis1,
);
let selected_verifier_data = self.select_verifier_data(
condition,
inner_verifier_data0,
inner_verifier_data1,
);

self.verify_proof::<C>(&selected_proof, &selected_verifier_data, inner_common_da
```

사용 예제

```
fn conditional_verification_example() -> Result<()> {
```

```rust
let config = CircuitConfig::standard_recursion_config();
// 두 개의 서로 다른 증명 생성
let proof0 = create_proof_type_a()?;
let proof1 = create_proof_type_b()?;
// 조건부 검증 회로 구성
let mut builder = CircuitBuilder::<F, D>::new(config);
let mut pw = PartialWitness::new();
// 조건 설정 (예: 특정 입력값에 따라)
let condition = builder.add_virtual_bool_target_safe();
pw.set_bool_target(condition, true)?; // proof0 검증
// 두 증명 모두 Target으로 추가
let proof_target0 = builder.add_virtual_proof_with_pis(&common_data);
let proof_target1 = builder.add_virtual_proof_with_pis(&common_data);
pw.set_proof_with_pis_target(&proof_target0, &proof0)?;
pw.set_proof_with_pis_target(&proof_target1, &proof1)?;
// 검증자 데이터 추가
let verifier_data_target0 = builder.add_virtual_verifier_data(cap_height);
let verifier_data_target1 = builder.add_virtual_verifier_data(cap_height);
// ... witness 설정 ...
// 조건부 검증
builder.conditionally_verify_proof::<C>(
condition,
&proof_target0,
```

```
&verifier_data_target0,
&proof_target1,
&verifier_data_target1,
&common_data,
);
let data = builder.build::<C>();
let proof = data.prove(pw)?;
data.verify(proof)?;
Ok(())
}
```

select 함수들

조건부 선택을 위한 헬퍼 함수들:

```
// 증명 선택
fn select_proof_with_pis(
&mut self,
b: BoolTarget,
proof0: &ProofWithPublicInputsTarget<D>,
proof1: &ProofWithPublicInputsTarget<D>,
) -> ProofWithPublicInputsTarget<D>

// 검증자 데이터 선택
```

```
pub fn select_verifier_data(
&mut self,
b: BoolTarget,
vk0: &VerifierCircuitTarget,
vk1: &VerifierCircuitTarget,
) -> VerifierCircuitTarget


// 해시 선택
pub(crate) fn select_hash(
&mut self,
b: BoolTarget,
h0: HashOutTarget,
h1: HashOutTarget,
) -> HashOutTarget
```

요약

재귀 증명은 plonky2의 강력한 기능으로, 다음을 가능하게 합니다:

1. **증명 압축**: 여러 증명을 하나로 집계
2. **증명 체인**: 증명을 연속적으로 연결
3. **조건부 검증**: 동적으로 검증 로직 선택
4. **확장성**: 대규모 시스템에서 효율적인 검증

핵심 구성 요소:

- `VerifierCircuitTarget` : 회로 내 검증자 데이터 표현
- `verify_proof` : 재귀 검증 로직 구현
- `add_virtual_proof_with_pis` : 증명을 회로 입력으로 추가
- `conditionally_verify_proof` : 조건부 검증 지원

재귀 증명을 통해 블록체인, 롤업, 대규모 계산 검증 등 다양한 응용이 가능합니다.

문서 네비게이션

- **이전**: 09. Square Root 예제 - Custom Generator 사용
- **다음**: 11. 성능 최적화 - 성능 최적화 기법
- **관련 문서**:
- 02. 회로 구성 프로세스 - 재귀 회로 구성
- 03. 증명 생성 프로세스 - 증명 생성 과정
- 04. 증명 검증 프로세스 - 검증 로직 이해
- 01. 아키텍처 개요 - 재귀 증명 구조